1 UNDERSTANDING ALGORITHMS

GENERAL VOCABULARY

construct a command to control the order/flow in which instructions are executed (e.g. sequences, selection, repetition)

It is important to be able to **construct** algorithms and be able to read them and follow their logic in solving particular problems.

LEARNING OBJECTIVES

- Understand what an algorithm is
- Understand what algorithms are used for
- Interpret algorithms as flowcharts, pseudocode and written descriptions
- Use and describe the purpose of arithmetic operators
- Understand how to code an algorithm in a high-level language

AN EXAMPLE OF AN ALGORITHM

SUBJECT VOCABULARY

unambiguous this means that the instructions cannot be misunderstood. Simply saying 'turn' would be ambiguous (i.e. unclear) because you could turn left or right. All instructions given to a computer must be unambiguous or it won't know what to do

sequence an ordered set of instructions

algorithm a precise method for solving a problem

An interactive map is a useful way to find a route between two locations. Figure 1.1 shows a route between two cities that was calculated by a mapping program.

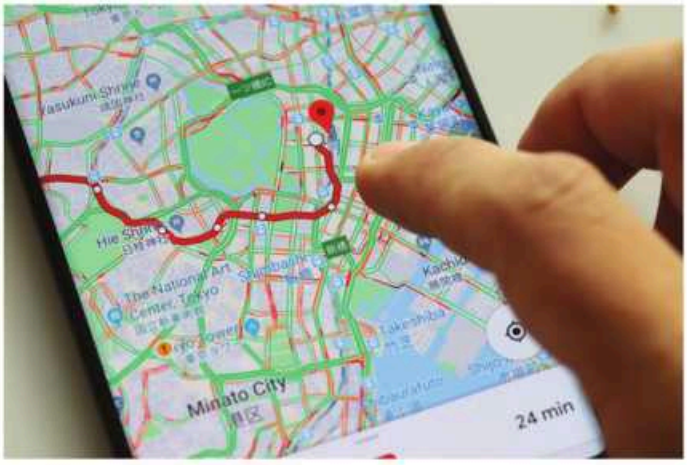
The route on this interactive map has been calculated using an algorithm.

- It is **unambiguous** in telling the driver exactly what to do, like 'turn left', 'turn right' or 'go straight'.
- It is a **sequence** of steps.
- It can be used again and will always provide the same result.
- It provides a solution to a problem, in this case, how to get from Beijing to Shanghai.

A solution to a problem with these characteristics is called an **algorithm**. Most problems have more than one solution, so different algorithms can be created for the same problem.

DID YOU KNOW?

The computer program that created the algorithm to map travel from Beijing to Shanghai was following an algorithm of its own – an algorithm ordering it how to create another algorithm!



▶ **Figure 1.1** A route calculated by a mapping program

SUCCESSFUL ALGORITHMS

There are three points to consider when deciding whether an algorithm is successful or not.

GENERAL VOCABULARY

outcome the final result of an action

consistency not changing; always the same

atlas a book of maps

- Accuracy – it must lead to the expected **outcome** (e.g. create a route from Beijing to Shanghai).
- **Consistency** – it must produce the same result each time it is run.
- Efficiency – it must solve the problem in the shortest possible time, using as few computer resources as possible. In this example, the mapping software is replacing a manual method. If it were no faster than looking in an **atlas**, then it would not be an improvement on the older method. Later in the unit there is a section on algorithms that are used to sort and search data. Some of these algorithms are more efficient than others and will sort the data far more quickly.

THE RELATIONSHIP BETWEEN ALGORITHMS AND PROGRAMS

Algorithms and programs are closely related, but they are not the same. An algorithm is a detailed design for a solution; a program is when that design is implemented.

SUBJECT VOCABULARY

high-level programming language a programming language that is similar to natural human language

This unit is all about algorithms. We look at how algorithms are implemented in **high-level programming languages** in Unit 2.

DISPLAYING AN ALGORITHM

We carry out many everyday tasks using algorithms because we are following a set of instructions to achieve an expected result, for example, making a cup of coffee. If we have performed the task many times before, we usually carry out the instructions without thinking. But if we are doing something unfamiliar, such as putting together a flat-pack chest of drawers, then we follow the instructions very carefully.

An algorithm can be expressed in different ways.

WRITTEN DESCRIPTIONS

A written description is the simplest way of expressing an algorithm. Here is an algorithm describing the everyday task of making a cup of instant coffee:



ALGORITHM FOR MAKING A CUP OF COFFEE

Fill kettle with water.
Turn on kettle.
Place coffee in cup.
Wait for water to boil.
Pour water into cup.
Add milk and sugar.
Stir.

SKILLS REASONING, PROBLEM SOLVING

ACTIVITY 1

GETTING TO SCHOOL

Produce a written description of an algorithm for getting to school. It should start with leaving home and end with arriving at school. For example, the algorithm could start with 'Walk to bus stop'.

Check your algorithm with other members of the group. Would your algorithm work for others? Are there any general statements that are common to all algorithms?

SUBJECT VOCABULARY

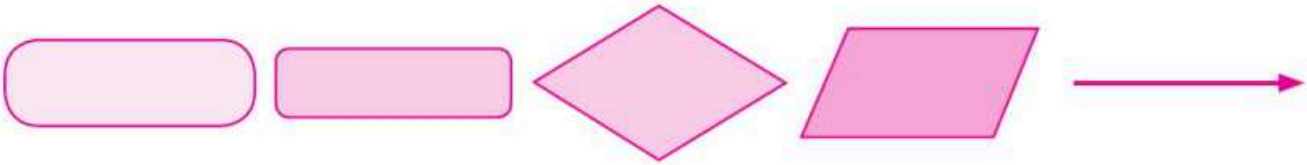
flowchart shows an algorithm as a diagram. Each step in the algorithm is represented by a symbol. Symbols are linked together with arrows showing the order in which steps are completed

FLOWCHARTS

Flowcharts can be used to show an algorithm as a diagram. They provide a more visual display.

There are special symbols that have to be used in a flowchart. You can't just make up your own, because nobody else would be able to follow your algorithm.

Figure 1.2 shows the flowchart symbols that should be used.

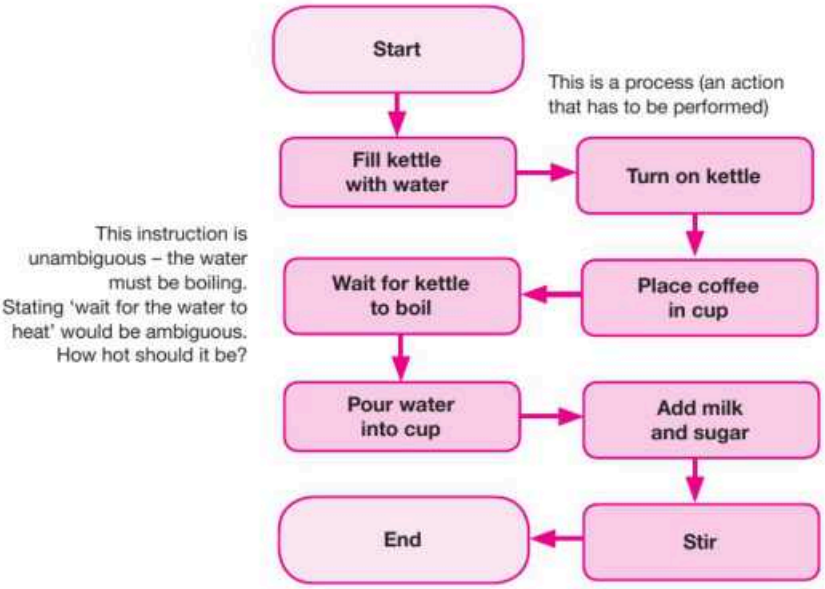


Indicates the start or end of an algorithm	Indicates a process to be carried out	Indicates a decision to be made	Indicates an input or output	Shows the logical flow of the algorithm

▲ **Figure 1.2** Flowchart symbols

The flowchart in Figure 1.3 is an alternative way of showing the algorithm for making a cup of coffee as a written description.

▶ **Figure 1.3** Flowchart of an algorithm to make a cup of coffee



SKILLS PROBLEM SOLVING

ACTIVITY 2

SCHOOL JOURNEY FLOWCHART

Display the 'journey to school' algorithm that you created in Activity 1 as a flowchart.

SKILLS PROBLEM SOLVING

ACTIVITY 3

BATH FLOWCHART

A student has created a written algorithm for preparing a bath. Working with a partner, display the following as a flowchart. You may need to change the order or add actions.

- Put in the plug.
- Fill the bath to the correct level.
- Check the temperature is OK.

GENERAL VOCABULARY

basis an important idea or fact that something is based on

SUBJECT VOCABULARY

pseudocode a structured, code-like language that can be used to describe an algorithm

developer a person whose job it is to create new software

logic the principles and reasoning underlying the constructs and elements to be applied in solving problems

The algorithms you have looked at so far are designed for humans to follow. Algorithms also form the **basis** of computer programs. Computers are mindless machines that simply do exactly what they are told. They follow a set of instructions, but they can carry out these instructions far more quickly than humans. That is why they are so useful.

PSEUDOCODE

In addition to flowcharts and written descriptions, algorithms can also be expressed in **pseudocode**. The pseudocode can be used to code the solution in an actual programming language.

It allows the **developer** to concentrate on the **logic** and efficiency of the algorithm without having to bother about the rules of any particular programming language. It is relatively straightforward to translate an algorithm written in pseudocode into any high-level programming language.

SKILLS RESEARCH

ACTIVITY 4

INVESTIGATING PSEUDOCODE

Different organisations or examination boards have their own unique versions of pseudocode. Investigate the Pearson Edexcel pseudocode that you will need for your International GCSE course and which will be used in this book.

EXAMPLE OF A SIMPLE ALGORITHM

To introduce the Pearson Edexcel pseudocode, here is a simple written algorithm that asks the user to input two numbers and then outputs the result of adding them together.

WRITTEN DESCRIPTION

ALGORITHM FOR ADDING TWO NUMBERS

Enter first number.
Enter second number.
Calculate total by adding first and second numbers.
Output total.

FLOWCHART



▲ **Figure 1.4** Flowchart showing the adding of two numbers

SUBJECT VOCABULARY

variable a 'container' used to store data. The data stored in a variable is referred to as a value. The value stored in a variable is not fixed. The same variable can store different values during the course of a program and each time a program is run

identifier a unique name given to a variable or a constant. Using **descriptive** names for variables makes code much easier to read

arithmetic operator an **operator** that performs a calculation on two numbers

GENERAL VOCABULARY

operator a character that represents an action, e.g. 'x' represents a multiplication and '/' a division

descriptive describing something clearly

PSEUDOCODE

ALGORITHM FOR ADDING TWO NUMBERS

```
SEND 'Please enter the first number' TO DISPLAY
RECEIVE firstNumber FROM KEYBOARD
SEND 'Please enter the second number' TO DISPLAY
RECEIVE secondNumber FROM KEYBOARD
SET total TO firstNumber + secondNumber
SEND total TO DISPLAY
```

The pseudocode gives clear step-by-step instructions that the computer will be expected to carry out. It also introduces some important programming concepts.

- The numbers entered by the user are stored in two **variables** with the **identifiers** `firstNumber` and `secondNumber`.
- The result of adding the numbers together is stored in the variable `total`.
- Text has to be placed in quotation marks (single or double) if it is to be displayed. For example, 'Please enter the first number' (or "Please enter the first number").
- Quotation marks are not used if a variable is to be displayed. If they were, the word 'total' in the last instruction would be displayed instead of the number it represents.
- **Arithmetic operators** are used to perform calculations. Table 1.1 shows the arithmetic operators.

ARITHMETIC OPERATORS

OPERATOR	FUNCTION	EXAMPLE
+	Addition: add the values together.	8 + 5 = 13 myScore1 + myScore2
–	Subtraction: subtract the second value from the first.	17 – 4 = 13 myScore1 – myScore2
*	Multiplication: multiply the values together.	6 * 9 = 54 numberBought * price
/	Real division: divide the first value by the second value and return the result including decimal places.	13 / 4 = 3.25 totalMarks/numberTests
DIV	**Quotient**: like division, but it only returns the whole number or *integer*.	13 DIV 4 = 3 totalMarks DIV numberTests
MOD	**Modulus**/modulo: this will return the remainder of a division.	13 / 4 = 3 remainder 1 Therefore 13 MOD4 = 1
^	Exponentiation: this is for 'to the **power** of'.	3 ^ 3 = 27 It is the same as writing 3^3

▲ **Table 1.1** Arithmetic operators

VARIABLES AND CONSTANTS

KEY POINT

When you chose a variable name, it should be descriptive of the data it will hold, e.g. distance or length. Long variable names are easier to misspell!

SUBJECT VOCABULARY

constant a 'container' that holds a value that never changes; like variables, constants have unique identifiers

GENERAL VOCABULARY

quotient a result found by dividing one quantity by another

modulus the remainder after the division of one number by another

power the small number written to the right and above another number to show how many times it should be multiplied by itself

Variables play an important role in algorithms and programming. The value stored by a variable can change as a program is running. Variables are extremely useful in programming because they make it possible for the same program to process different sets of data.

A **constant** is the opposite of a variable. It is a 'container' that holds a value that always stays the same. Constants are useful for storing fixed information, such as the value of pi, the number of litres in a gallon or the number of months in a year.

Each variable and constant in an algorithm has to have a unique identifier. It is important to choose descriptive names for identifiers. This will make your code much easier to read. For example, a variable to hold a user's first name could be given the identifier `firstName` to show the data it contains. If it were given the identifier `X` instead, it wouldn't clearly show what data it contained.

NAMING CONVENTIONS FOR VARIABLES AND CONSTANTS

It is sensible to write identifiers in the same way throughout an algorithm. A common method is to use 'camel case' for compound words (e.g. `firstName`, `secondName`) with no space between words and the second word starting with a capital letter. Alternatively, you could capitalise the first letter of both words, e.g. `FirstName`, `SecondName`, or separate the words with an underscore, e.g. `first_name`, `second_name`, known as 'snake case'.

SKILLS PROBLEM SOLVING, ANALYSIS

ACTIVITY 5

WRITING ALGORITHMS IN PSEUDOCODE

Here is a written description of an algorithm:

Enter the first number.
Enter the second number.
The third number is equal to the first number multiplied by the second number.
Display the third number.

Express this algorithm in pseudocode.

KEY POINT

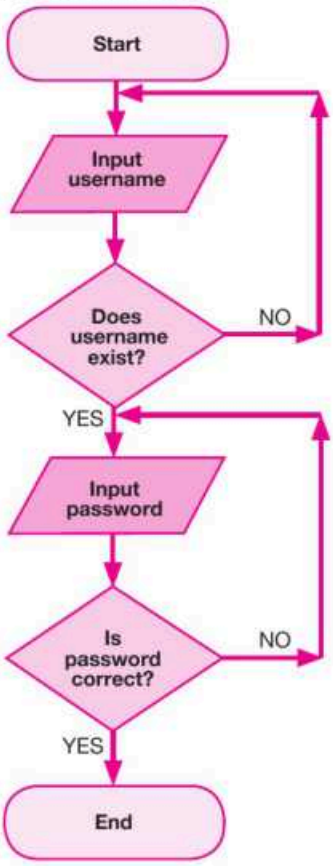
Whichever method you use for naming conventions, you must use it consistently and not keep switching between the different methods.

SKILLS PROBLEM SOLVING

ACTIVITY 6

WRITTEN DESCRIPTIONS OF ALGORITHMS

This algorithm is displayed as a flowchart.



▲ **Figure 1.5** Flowchart of an algorithm

Produce a written description of this algorithm.

EXTEND YOUR KNOWLEDGE

When you enter a search term into Google®, a list of links to websites is returned. But why are they presented in that particular order? With a partner, research the `PageRank` algorithm that Google uses to rate the importance of websites and write a short report about your findings.

CHECKPOINT

Strengthen

SKILLS CRITICAL THINKING

S1 Produce a written description of an algorithm for borrowing a book from the library.

SKILLS DECISION MAKING

S2 What is the function of each of the seven arithmetic operators?

SKILLS REASONING

S3 What is a variable? Why are they useful?

SKILLS CRITICAL THINKING

S4 What is the difference between a variable and a constant?

Challenge

SKILLS CRITICAL THINKING, PROBLEM SOLVING

C1 Produce a flowchart describing an algorithm for making a cheese sandwich.

SKILLS PROBLEM SOLVING

C2 Write an algorithm expressed in pseudocode that receives three numbers from the keyboard, then calculates and displays the average.

How confident do you feel about your answers to these questions? If you're not sure you answered them well, try the following activities again.

- For S1 re-read 'The relationship between algorithms and programs'.
- For S2 study Table 1.1.
- For S3 and S4 look again at 'Variables and constants'.

SUMMARY

- An algorithm is a precise method for solving a problem.
- Algorithms can be displayed as written descriptions, flowcharts and in pseudocode.
- Pseudocode is a **structured**, code-like language.
- Pseudocode is translated into program code.
- Arithmetic operators are used in calculations.
- Variables and constants are 'containers' for storing data. The value stored in a variable can change, whereas the value of a constant never changes.
- Selecting descriptive names for identifiers makes code easier to read.

GENERAL VOCABULARY

structured organised so that the parts work well together